

Mikrocontroller: C-Coding Style

Dieses Dokument beschreibt den bevorzugten Coding-Stil für Mikrocontroller bei Programmen in der Programmiersprache C, also der in dieser Anwendung (fast) einzigen.

Dateien - Module - Namen

1. Dateinamen sind grundsätzlich auf englisches Vokabular zurückzuführen. Wie alles andere auch.
2. Die Funktionen und Variablen inklusive deren Deklaration jeder einzelnen Gruppe (Modul) sind in einer Datei implementiert. Diese Datei trägt dann den Namen "module.c" und beinhaltet auch die Deklarationen, wodurch eine zusätzliche Header-Datei eingespart werden kann. Wenn nur der Header-Teil dieses Moduls eingebunden werden soll, muss vor dem "#include "module.c"" noch das Makro "HEADER_MODULE" definiert werden.
Anmerkung: Dies kann, wenn vom Kunden anders gewünscht, natürlich auch im klassischen Stil mit ".h"- und ".c"-Datei erfolgen.
3. Die Datei, welche das Hauptprogramm beinhaltet, heißt "main.c".
4. Einzelne, benötigte Funktionen, welche kein eigenes Modul verlangen, werden in einem Modul für diverse Funktionalität zusammengefasst (zum Beispiel: "misc.c").

Deklarationen - Definitionen

1. Im Header-Teil erfolgen die Deklarationen der öffentlichen Komponenten in der Reihenfolge: Makros, Datentypen, Variablen, Funktionen.
Anmerkung: Wenn es sinnvoll ist, ein Makro kurz vor einer Variablen oder Funktion zu definieren, wird dies auch so gemacht.
2. Die öffentlichen Variablen und Funktionen eines Moduls beginnen immer mit einem Kürzel in Blockbuchstaben für das Modul gefolgt von '_'. Danach folgt der Name in Upper Camel Case.
Beispiel: MOD_GetData(void), Kürzel "MOD_" für "module.c"
3. Dieses Kürzel sollte auf maximal vier Zeichen, vorzugsweise zwei oder drei, beschränkt sein.
Anmerkung: Den kompletten Modulnamen auszuschreiben macht die Namen nur unnötig lang, denn da Mikrocontroller-Programme nicht aus hunderten Modulen bestehen, sollten Abkürzungen ausreichen.
4. Jede einzelne Komponente ist Doxygen-dokumentiert, egal ob öffentlich oder privat.
5. Private Variablen und Funktionen können auf das Modul-Kürzel zu Beginn verzichten. Das Modul-Kürzel wird manchmal trotzdem verwendet, vor allem dann, wenn Doppeldeutigkeiten im Debugger entstehen könnten, weil gleich benannte, private Variablen in mehreren Modulen existieren.
6. Die Namen von Strukturen, Unions und andere Datentypen haben kein zusätzliches Prä- oder Suffix.
Anmerkung: Es sollte in der Verwendung ohnehin klar sein worum es sich handelt. Wenn dies nicht so ist, sollte die Sinnhaftigkeit des Datentypen hinterfragt werden.
7. Elemente in Strukturen werden ohne beschreibenden Variablennamen benannt, solange die Bedeutung offensichtlich ist.

Beispiel: Der Datentyp "Coordinate" ist eine Struktur und beinhaltet drei Variablen mit den Namen 'x', 'y' und 'z'.

Funktionen - Programmzeilen

1. Zu Beginn jeder Funktion werden alle darin verwendeten Variablen definiert. Definitionen innerhalb der Funktion werden ausnahmslos vermieden.
2. Danach folgt eine Leerzeile, nach der mit den Befehlen begonnen wird.
3. Auch wenn return keine Funktion ist, wird der Rückgabewert immer in Klammer gesetzt (wie es ganz alte Compiler früher verlangt haben).
4. Das Beginn-Symbol '{' wird in der nächsten Zeile mit gleichen Einrückung wie der Befehl bzw. Funktionsnamen in der Zeile davor geschrieben.
5. Der erste Befehl steht in derselben Zeile wie das '{'-Zeichen.
6. Die Einrückung beträgt prinzipiell zwei Leerzeichen. Tabulatoren sind tabu.
7. Wenn Befehle trivial sind, können auch mehrere in einer Zeile stehen.
8. Zum Kommentieren wird grundsätzlich nur das Kommentar im C-Stil verwendet:
"/ * blabla * /" (Man denke an MISRA-C.)
9. Falls ein Kommentar im C++-Stil eingesetzt wird ("// blabla"), markiert das eine temporäre Zwischenlösung, die wieder bereinigt werden soll.

Variablen - Zuweisungen - Operationen

1. Lokale Variablen in Funktionen, welche nur einen Buchstaben lang sind, werden, wenn sie mit dem einzelnen Buchstaben den Inhalt ausreichend genau beschreiben, verwendet.
Zum Beispiel: 'i', 'j', 'k', 'l', 'm', 'n' für Indizes, Laufvariablen und Anzahl; 'd', 'f' für temporäre Fließkomma-Variablen; 'x', 'y', 'z' für kartesische Koordinaten; 'c' für ASCII-Zeichen bzw. 's' Zeichenketten, ...
2. Einbuchstabile Variablen sind auch erlaubt, wenn der Geltungsbereich nur eine einfach überschaubare Anzahl von Zeilen beträgt.
Zum Beispiel: Kurze, überschaubare Funktionen, die nicht einmal einen halben Bildschirm lang sind.
3. Auf Datentypen wie uint16_t und ähnliche wird verzichtet, da diese ohne Syntax-Highlighting nur die Verständlichkeit vermindern. Stattdessen werden die Makros "i16" und "i32" verwendet, um je nach Zielsystem 16-Bit- und 32-Bit-Typen zu beschreiben. Die einzige Ausnahme ist, wenn Bibliotheksfunktionen anderer solche Typen verlangen.
4. Bei ganzzahligen Datentypen, deren Information in der Zahl steckt, werden vorzugsweise signed-Variablen benutzt. Unsigned-Variablen werden nur dann verwendet, wenn die Information in den einzelnen Bits steckt (zum Beispiel: Bitfelder, Farbwerte, ...).
5. Die ungarische Notation wird nicht verwendet.
Anmerkung: Ungarische Notation ist höflich geschrieben sehr entbehrlich.
6. Für ganzzahligen Werte werden Variablen mit dem natürlichen Datentyp (Registerbreite des Mikrocontrollers) verwendet, also "int".
Anmerkung: Ein Mikroprozessor sollte nicht zu kleineren Datentypen gezwungen werden, da dies in der Ausführungsgeschwindigkeit nicht von Vorteil ist und dabei zu keinem signifikanten Speichervorteil führt.
7. Angeblich der Lesbarkeit dienliche Leerzeichen zwischen Operationen und Zuweisungen werden nicht eingesetzt.
Anmerkung: Dadurch werden vor allem intensivere, mathematische Berechnungen deutlich länger und damit unübersichtlicher. Aber das ist Ansichtssache.

8. Bei nicht offensichtlichen Prioritäten von Operatoren wird lieber die eine oder andere Klammerung zu viel vorgenommen.

Compiler

1. Warnungen werden grundsätzlich als Fehler betrachtet und sind somit nicht erlaubt.
2. Falls eine Warnung in einer Datei, die von einer Bibliothek kommt, entsteht, so ist diese zu akzeptieren.
3. Während der Entwicklung werden Compiler-Optimierungen nur beschränkt eingesetzt, da diese zu Debugging-Problemen führen können und werden.
4. Ziel der Entwicklung ist es ohnehin, ohne exzessives Optimieren ein gut funktionierendes Programm zu implementieren.

Grundsätzliches zum Thema Zusammenarbeit

1. Bibliotheken und zugekaufter Quellcode werden nicht verändert.
Anmerkung: Dies rächt sich üblicherweise bei Updates dieser.
2. Falls das aus welchen Gründen auch immer notwendig wird, dann muss diese Veränderung im Code klar gekennzeichnet werden. Es ist auch zu empfehlen, dass diese Veränderungen an den Lieferanten weitergegeben werden bzw. sollten die gewünschten Anpassungen vom Lieferanten selbst durchgeführt werden.
3. Beleidigungen in Kommentaren im Quellcode oder Versionsverwaltungssystem sind zu unterlassen.
4. Andererseits darf man auch die Berichtigung eines Fehlers bzw. den Hinweis auf einen durch eine andere Person nicht als persönlichen Angriff betrachten.
5. Auch wenn man zusammenarbeitet, ein Herumpfuschen im Quellcode des anderen zur gefühlten Behübschung oder Verbesserung (vor allem in getesteten, stabil funktionierenden Programmteilen), bleibt ein Herumpfuschen.
6. Das Herauslöschten des tatsächlichen Autors, Veränderung des Coding-Stils mit nachträglichem eigenem Eintragen als Autor ist eine äußerst fragwürdige und verwerfliche Vorgangsweise.
7. Zusammengefasst sollte es einen gegenseitigen Respekt vor der Arbeit und des Stils des anderen geben.